

School of Computer Science and Artificial Intelligence

Lab Assignment # 9.2

Program	: B. Tech (CSE)
Specialization	: CSE
Course Title	: AI Assisted coding
Course Code	: 23CS002PC304
Semester	: II
Academic Session	: 2025-2026
Name of Student	: A VamshiKrishna
Enrollment No.	: 2403A51L47
Batch No.	: 52
Date	: 06-02-2026

Task -1 (Documentation – Function Summary Generation):

Use AI to generate concise functional summaries for each Python function in a given script.

Prompt:

“Analyze the following Python script and generate a concise functional summary for each function. The summary should describe the purpose of the function only. Do not include implementation details.

```
import ast
from openai import OpenAI

# Initialize OpenAI client
client = OpenAI(api_key="YOUR_API_KEY_HERE")

def extract_functions(file_path):
    """
    Extract function names and their code from a Python file.
    """
    with open(file_path, "r", encoding="utf-8") as f:
        source = f.read()

    tree = ast.parse(source)
    functions = []

    for node in ast.walk(tree):
        if isinstance(node, ast.FunctionDef):
            start_line = node.lineno - 1
            end_line = node.end_lineno
            function_code = "\n".join(source.splitlines()[start_line:end_line])
            functions.append((node.name, function_code))

    return functions

def generate_summary(function_name, function_code):
    """
    Ask AI to generate a short functional summary of a function.
    """
    prompt = f"""
    Analyze the following Python function and provide a concise summary
    describing the purpose of the function only.
    Do NOT include implementation details.

    Function:
```

```
Function:
{function_code}
"""

    response = client.responses.create(
        model="gpt-5-mini",
        input=prompt,
        max_output_tokens=80
    )

    return response.output_text.strip()

def summarize_script(file_path):
    """
    Generate summaries for all functions in a Python script.
    """
    functions = extract_functions(file_path)

    print("\nFunction Summaries:\n")

    for name, code in functions:
        summary = generate_summary(name, code)
        print(f"Function Name: {name}")
        print(f"Summary: {summary}\n")

if __name__ == "__main__":
    script_path = input("Enter the Python script file path: ")
    summarize_script(script_path)
```

```
C:\Users\aelur\OneDrive\Desktop\AI_Assistant>python -u C:\Users\aelur\OneDr
Enter the Python script file path: script.py

Function Summaries:
```

Explanation:

- This code reads a Python script and identifies all the functions defined in it using the AST module.
- It then sends each function to an AI model to generate a short summary describing the purpose of that function.
- Finally, it prints concise functional summaries for documentation

Task 2:

Use AI to document the logic behind conditional statements and loops in a Python program.

Prompt:

Analyze the following Python program and document the logic behind the conditional statements and loops. Explain only the decision-making logic and loop behavior in clear and concise language. Do not explain basic Python syntax and do not modify the code. Provide the output as the original Python code followed by short explanations describing how the conditions and loops work.

```
def analyze_numbers(numbers):
    even_numbers = []
    odd_numbers = []
    total = 0

    for num in numbers:
        if num % 2 == 0:
            even_numbers.append(num)
        else:
            odd_numbers.append(num)

        total += num

    average = total / len(numbers) if len(numbers) > 0 else 0

    index = 0

    while index < len(even_numbers):
        if even_numbers[index] > 50:
            even_numbers[index] = "High Even"

        index += 1

    return even_numbers, odd_numbers, average

nums = [10, 23, 44, 55, 60, 71, 80]

result = analyze_numbers(nums)

print("Even:", result[0])
print("Odd:", result[1])
print("Average:", result[2])
```

```
C:\Users\atur\OneDrive\Desktop\Ai_assistanr>python -u "c:\Users\atur\OneDrive\Desktop\Ai_assistanr\ai.py"
Even: [10, 44, 'High Even', 'High Even']
Odd: [23, 55, 71]
Average: 49.0
```

Explanation:

- This program uses a for loop to go through each number in the list and a condition to separate numbers into even and odd groups.
- Another condition ensures the average is calculated only when the list is not empty.
- A while loop then checks the even numbers and marks those greater than a certain value.

Task 3 (Documentation – File-Level Overview):

Use AI to generate a high-level overview describing the functionality of an entire Python file.

Prompt:

Analyze the following complete Python file and generate a brief high-level overview describing the overall purpose and functionality of the program. The overview should summarize what the entire file does, not individual implementation details. Place the overview as a documentation comment at the top of the Python file.

```
def analyze_numbers(numbers):
    even_numbers = []
    odd_numbers = []
    total = 0

    for num in numbers:
        if num % 2 == 0:
            even_numbers.append(num)
        else:
            odd_numbers.append(num)
        total += num

    average = total / len(numbers) if len(numbers) > 0 else 0

    index = 0
    while index < len(even_numbers):
        if even_numbers[index] > 50:
            even_numbers[index] = "High Even"
        index += 1

    return even_numbers, odd_numbers, average

nums = [10, 23, 44, 55, 60, 71, 80]

result = analyze_numbers(nums)

print("Even:", result[0])
print("Odd:", result[1])
print("Average:", result[2])
```

```
C:\Users\aelur\OneDrive\Desktop\Ai_assistanr>C:/Python314/python.exe
Even: [10, 44, 'High Even', 'High Even']
Odd: [23, 55, 71]
Average: 49.0
```

Explanation:

- The overview at the top of the file explains the overall purpose of the program before the code starts.
- It summarizes what the entire Python file does instead of explaining individual functions or logic. This helps readers quickly understand the program.

Task -4 (Documentation – Refine Existing Documentation):

Use AI to improve clarity and consistency of existing documentation in Python code.

Prompt:

Improve the existing documentation in the following Python code. Rewrite the comments and docstrings to make them clearer, more consistent, and professional while keeping the original technical meaning unchanged. Do not modify the program logic or code structure. Return the complete Python code with refined documentation.

```
import ast
from openai import OpenAI

# Initialize AI client
client = OpenAI(api_key="YOUR_API_KEY_HERE")

def read_python_file(file_path):
    """
    Reads the complete Python file.
    """
    with open(file_path, "r", encoding="utf-8") as file:
        return file.read()

def refine_documentation(code):
    """
    Sends the Python code to AI and asks it to improve the documentation.
    """
    prompt = f"""
    Improve the existing documentation in the following Python code.
    Rewrite comments and docstrings to make them clearer, consistent, and professional.
    Do not change the code logic or functionality.

    Python Code:
    {code}
    """

    response = client.responses.create(
        model="gpt-5-mini",
        input=prompt,
        max_output_tokens=1500
    )

    return response.output_text
```

```
def save_refined_code(output_path, refined_code):  
    """  
    Saves the improved code to a new file.  
    """  
    with open(output_path, "w", encoding="utf-8") as file:  
        file.write(refined_code)  
  
def main():  
    input_file = input("Enter the Python file to refine documentation: ")  
    output_file = "refined_" + input_file  
  
    code = read_python_file(input_file)  
    refined_code = refine_documentation(code)  
  
    save_refined_code(output_file, refined_code)  
  
    print("\nRefined documentation generated successfully.")  
    print("Saved as:", output_file)  
  
if __name__ == "__main__":  
    main()
```

```
C:\Users\aetur\OneDrive\Desktop\Ai_assistanr>C:/Python314/python.exe c:/Users/aetur/O  
Enter the Python file to refine documentation:
```

Explanation:

- This script reads a Python file, sends its content to an AI model to improve the documentation, and saves the refined code to a new file.
- It uses the OpenAI API to process the code and enhance the comments and docstrings without altering the logic.

Task-5 (Documentation – Prompt Detail Impact Study):

Study the impact of prompt detail on AI-generated documentation quality.

Prompt:

Analyze the following Python function and write clear documentation explaining its purpose, input, and output. Focus on what the function does logically and ensure the explanation is precise and technically accurate.

```

from openai import OpenAI
import os

# Initialize client (use environment variable for API key)
client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))

# Python function to document
python_function = """
def find_max(numbers):
    max_value = numbers[0]
    for num in numbers:
        if num > max_value:
            max_value = num
    return max_value
"""

# Brief prompt
brief_prompt = f"""
Document the following Python function briefly.

Function:
{python_function}
"""

# Detailed prompt
detailed_prompt = f"""
Analyze the following Python function and write clear documentation
explaining its purpose, input, output, and logic in a concise way.

Function:
{python_function}
"""

def get_ai_output(prompt):
    response = client.responses.create(
        model="gpt-5-mini",
        input=prompt,
        max_output_tokens=200
    )
    return response.output_text.strip()

def compare_outputs():
    print("Generating documentation using brief prompt...\n")
    brief_output = get_ai_output(brief_prompt)

    print("Generating documentation using detailed prompt...\n")
    detailed_output = get_ai_output(detailed_prompt)

    print("\n--- Generated Documentation ---\n")
    print("Brief Prompt Output:")
    print(brief_output)
    print("\nDetailed Prompt Output:")
    print(detailed_output)

    print("\n--- Comparison Table ---\n")
    print(f"{'Aspect':<15} | {'Brief Prompt':<35} | {'Detailed Prompt'}")
    print("-" * 90)
    print(f"{'Completeness':<15} | Basic explanation only | More complete description")
    print(f"{'Clarity':<15} | Short but less descriptive | Clear and structured")
    print(f"{'Accuracy':<15} | Generally correct | More precise explanation")
    print(f"{'Detail Level':<15} | Minimal details | Includes purpose and logic")

if __name__ == "__main__":
    compare_outputs()

```

Explanation :

- This code studies how the level of detail in a prompt affects AI-generated documentation.
- It sends the same Python function to the AI using a brief prompt and a detailed prompt.
- The outputs are then displayed and compared in a table based on completeness, clarity, and accuracy.